

KANTONSSCHULE REUSSBÜHL

ANFORDERUNGSANALYSE ZUR MATURAAARBEIT

Entwicklung eines lizenzfreien Funkprotokolls zur Schliessung der Bandbreiten-Reichweiten-Lücke zwischen WLAN und LoRa

Valentin Seeholzer

K23a

Betreut durch:
Herr Daniel Zurmühle

Hinweis zur Dokumentation: Dieses Dokument definiert die funktionalen und nicht-funktionalen Anforderungen an das zu entwickelnde System sowie die Stakeholder und Zeitplan.

20. April 2026

Inhaltsverzeichnis

1	Ziel	3
2	Anforderungen	3
2.1	Grundfunktionen (Must-have)	3
2.2	Erweiterte Funktionen (Should-have)	3
2.3	Zusätzliche Funktionen (Nice-to-have)	4
3	Technische Voraussetzungen	4
4	Stakeholder	4
5	Dokumentation	4
6	Zeitplan und Priorisierung	6

1 Ziel

Das Ziel dieser Arbeit ist es, ein eigenes Funkprotokoll für ein Software Defined Radio (SDR) zu programmieren und in der Praxis zu testen. Dabei wird die Zuverlässigkeit von LoRa (Chirp Spread Spectrum) mit der Effizienz von QPSK (Quadraturphasesumtastung) kombiniert, um im 2.4 GHz-Band eine hohe Datenrate von etwa 4 Mbps zu erreichen.

Das entwickelte Programm dient dazu, dieses theoretische Konzept auf echter Hardware (einem Zynq-7020 SoC mit einem AD9363 Funk-Transceiver) zu testen. Mithilfe der Bibliothek `libiio` wird das System unter realen Funkbedingungen (Rauschen, Dämpfung, Phasenverschiebung) betrieben und anschließend statistisch ausgewertet.

2 Anforderungen

2.1 Grundfunktionen (Must-have)

- ☐ **[FA-10] Hardware-Anbindung:** Das System muss in der Lage sein, eine Verbindung zum AD9363 Transceiver herzustellen und Parameter wie Trägerfrequenz (2.4 GHz), Bandbreite und Samplerate dynamisch zu konfigurieren.
- ☐ **[FA-20] Modulator (Sender):** Das System muss einen digitalen Bitstrom in hybride CSS/QPSK-Symbole umwandeln und als komplexe I/Q-Werte an den Hardware-DAC (Digital-to-Analog Converter) senden können.
- ☐ **[FA-30] Demodulator (Empfänger):** Das System muss empfangene I/Q-Samples der Hardware per inverser FFT (Dechirp) verarbeiten und sowohl den Frequenzversatz (LoRa-Wert) als auch die Phasenlage (QPSK-Wert) extrahieren.
- ☐ **[FA-40] Fehlerkorrektur (LDPC):** Das System muss die gesendeten Daten mittels Low-Density Parity-Check (LDPC) Codes absichern, um entstandene Bitfehler korrigieren zu können.
- ☐ **[FA-50] Synchronisation:** Das Empfängerskript muss das Startsignal eines Pakets (Präambel) im Rauschen selbstständig erkennen.
- ☐ **[FA-60] Datenerfassung:** Die demodulierten Daten müssen gespeichert und zeitlich gegliedert werden, um sie im Nachhinein präzise auswerten zu können.

2.2 Erweiterte Funktionen (Should-have)

- ☐ **[FA-70] Soft-Decision Decoding:** Der Demodulator sollte Wahrscheinlichkeiten (Log-Likelihood Ratios, LLRs) anstelle von harten Bits an den LDPC-Decoder übergeben, um die Empfindlichkeit des Systems zu steigern.
- ☐ **[FA-80] Statistische Auswertung:** Das Programm sollte die Paketfehlerrate (PER) und Bitfehlerrate (BER) bei verschiedenen Signal-Rausch-Verhältnissen (SNR) messen und in der Konsole ausgeben. (Simulation oder Auswertungsprogramm)

2.3 Zusätzliche Funktionen (Nice-to-have)

- **[FA-90] Hardware-Beschleunigung:** Auslagerung des rechenintensiven Dechirp-Prozesses (FFT) in die programmierbare Logik (FPGA) des Zynq-7020 mittels Vitis HLS, um USB-/Netzwerk-Engpässe zu umgehen.
- **[FA-100] MIMO-Nutzung:** Parallele Nutzung der zwei RX- und TX-Antennen des AD9363 (Spatial Multiplexing), um die Datenrate potenziell zu verdoppeln.

3 Technische Voraussetzungen

- **Hardware:** Ein SDR-Board basierend auf dem Xilinx Zynq-7020 Chip, gekoppelt mit einem Analog Devices AD9363 (oder AD9361) RF-Transceiver. Zusätzlich werden geeignete 2.4 GHz Antennen benötigt.
- **Software & Bibliotheken:** Die Programmierung erfolgt in Python 3.x. Für die digitale Signalverarbeitung kommen `numpy` und `scipy` zum Einsatz. Die Fehlerkorrektur nutzt die Bibliothek `ldpc`, und die Hardware-Kommunikation wird über `pyadi-iiio` (`libiiio`) abgewickelt.
- **Umgebung:** Das Skript läuft entweder direkt auf dem Linux-System des Zynq-Prozessors oder wird über eine Netzwerk-/USB-Verbindung von einem Host-PC aus gesteuert.

4 Stakeholder

Die Software wird hauptsächlich für Forschungszwecke entwickelt, um ein vergleichbares System als experimentelle Alternative zum klassischen LoRa-Verfahren oder zu WLAN-Systemen mit OFDM-Verfahren zu evaluieren. Als primäre Zielgruppe kommen daher Lehrpersonen, Forscher und Studierende im Bereich der Nachrichtentechnik in Frage, die das Potenzial hybrider Modulationsverfahren auf SDR-Hardware in der Praxis sehen möchten.

Als sekundäre Zielgruppe sind Entwickler von IoT-Systemen anzusehen, die an Protokollen interessiert sind, die eine höhere Datenrate als Standard-LoRa bei gleichzeitig hoher Robustheit bieten. Bei der Definition der Anforderungen habe ich mich an wissenschaftlichen Standards der digitalen Signalverarbeitung orientiert und darauf geachtet, dass die Ergebnisse fundiert mit anderen Systemen verglichen werden können.

5 Dokumentation

Die gesamte Dokumentation meiner Maturaarbeit wird transparent und nachvollziehbar erstellt. Dazu nutze ich die Plattform GitHub, auf der ich sowohl den Programmcode als auch alle geschriebenen Texte (wie diese Anforderungsanalyse) öffentlich speichere. Alle Dokumente werden mit dem Textsatzsystem $\text{\LaTeX}$ geschrieben, um ein professionelles Layout zu garantieren.

Damit andere Personen meine Ergebnisse später verstehen und sogar selbst nachmessen können, schreibe ich eine genaue technische Dokumentation. Darin erkläre ich:

- Wie das von mir entwickelte Funk-Protokoll physikalisch funktioniert.
- Wie die einzelnen Programm-Teile aufgebaut sind.
- Wie die Software mit dem Funk-Gerät (der Hardware) zusammenarbeitet.

Ein klassisches "Benutzerhandbuch" für Anfänger ist für dieses Projekt nicht sinnvoll, da es sich um ein technisches Experiment handelt. Stattdessen schreibe ich eine kurze, genaue Anleitung (eine sogenannte "Readme-Datei"). Darin steht Schritt für Schritt beschrieben, wie man die Hardware anschließt, das Test-Programm startet und Einstellungen (wie zum Beispiel das Rauschen) verändert.

Da meine Arbeit auf GitHub für alle sichtbar ist, kann jeder verfolgen, wie sich das Projekt entwickelt hat. Sobald ich einen wichtigen Meilenstein erreiche (zum Beispiel wenn das System schneller oder fehlerfreier wird), passe ich die Dokumentation und die Anleitung direkt an. So bleibt das gesamte Projekt immer auf dem neuesten Stand.

6 Zeitplan und Priorisierung

Tabelle 1: Detaillierter Zeitplan der Maturaarbeit (Ziel: 01.09.2026)

Phase	Meilensteine / Aufgaben	Zeitraum
Phase 1: Implementierung	SDR & Code-Optimierung Fertigstellung Python-Programm (Sync, Präambel) Optimierung Timing & QPSK-Korrektur Code-Bereinigung & Coding-Standards	April – Mai '26
Phase 2: Analyse	Messungen & Theorie-Vertiefung Durchführung der Teilauswertungen Ausarbeitung der FFT-Theorie Erstellung finaler Diagramme & Grafiken	Juni '26
Phase 3: Rohfassung I	Theorie & Methodik schreiben Theorie I: Nachrichtentechnik & Kodierung Theorie II: HF-Technik & Regulatorik Methoden: Software- & Hardware-Setup	Juli '26
Phase 4: Rohfassung II	Resultate & Abschluss schreiben Auswertung der Messresultate Diskussion, Schlussfolgerung & Ausblick Meilenstein: Inhaltliche Fertigstellung	August '26 01.09.2026
Phase 5: Revision	Feinschliff & Abgabe Überarbeitung (Stil, Orthografie) Finalisierung Anhang & Quellenverzeichnis Druck & Bindung Abgabefrist	Sept. – Okt. '26 08.10.2026